



CredShields

Smart Contract Audit

May 8, 2026 • CONFIDENTIAL

Description

This document details the process and result of the Smart Contract audit performed by CredShields Technologies PTE. LTD. on behalf of SeerDEX between May 6, 2026, and May 7, 2026. A retest was performed on May 8, 2026.

Author

Shashank (Co-founder, CredShields) shashank@CredShields.com

Reviewers

Aditya Dixit (Research Team Lead), Shreyas Koli (Auditor), Sanket Salavi (Auditor), Prasad Kuri (Auditor), Neel Shah (Auditor)

Prepared for

SeerDEX



Table of Contents

Table of Contents	2
1. Executive Summary -----	3
State of Security	4
2. The Methodology -----	5
2.1 Preparation Phase	5
2.1.1 Scope	5
2.1.2 Documentation	5
2.1.3 Audit Goals	6
2.2 Retesting Phase	6
2.3 Vulnerability classification and severity	6
2.4 CredShields staff	8
3. Findings Summary -----	9
3.1 Findings Overview	9
3.1.1 Vulnerability Summary	9
5. Bug Reports -----	10
Bug ID #L001[Acknowledged]	10
Outdated Pragma	10
Bug ID#I001[Fixed]	12
Dead Code	12
6. The Disclosure -----	13



1. Executive Summary -----

SeerDEX engaged CredShields to perform a smart contract audit from May 6, 2026, to May 7, 2026. During this timeframe, 2 vulnerabilities were identified. **A retest was performed on May 8, 2026, and all the bugs have been addressed.**

During the audit, 0 vulnerabilities were found with a severity rating of either High or Critical. These vulnerabilities represent the greatest immediate risk to "SeerDEX" and should be prioritized for remediation, and fortunately, none were found.

The table below shows the in-scope assets and a breakdown of findings by severity per asset. Section 2.3 contains more information on how severity is calculated.

Assets in Scope	Critical	High	Medium	Low	info	Gas	Σ
SeerDEX Token Contract	0	0	0	1	1	0	2

Table: Vulnerabilities Per Asset in Scope

The CredShields team conducted the security audit to focus on identifying vulnerabilities in SeerDEX Token Contract's scope during the testing window while abiding by the policies set forth by SeerDEX's team.



State of Security

To maintain a robust security posture, it is essential to continuously review and improve upon current security processes. Utilizing CredShields' continuous audit feature allows both SeerDEX's internal security and development teams to not only identify specific vulnerabilities but also gain a deeper understanding of the current security threat landscape.

To ensure that vulnerabilities are not introduced when new features are added, or code is refactored, we recommend conducting regular security assessments. Additionally, by analyzing the root cause of resolved vulnerabilities, the internal teams at SeerDEX can implement both manual and automated procedures to eliminate entire classes of vulnerabilities in the future. By taking a proactive approach, SeerDEX can future-proof its security posture and protect its assets.



2. The Methodology -----

SeerDEX engaged CredShields to perform a SeerDEX Token Contract audit. The following sections cover how the engagement was put together and executed.

2.1 Preparation Phase

The CredShields team meticulously reviewed all provided documents and comments in the smart contract code to gain a thorough understanding of the contract's features and functionalities. They meticulously examined all functions and created a mind map to systematically identify potential security vulnerabilities, prioritizing those that were more critical and business-sensitive for the refactored code. To confirm their findings, the team deployed a self-hosted version of the smart contract and performed verifications and validations during the audit phase.

A testing window from May 6, 2026, to May 7, 2026, was agreed upon during the preparation phase.

2.1.1 Scope

During the preparation phase, the following scope for the engagement was agreed upon:

IN SCOPE ASSETS

- **Audited Scope:**
<https://github.com/PredexDev/Predex-Token-Contract/tree/617bec519ee74ecfd23d030546993cc496433888>
- **Retested Scope:**
<https://github.com/PredexDev/Predex-Token-Contract/tree/6bc74578ad273696ed7cf5553b150e4671e4885c>
- **Deployed Contract:**
<https://etherscan.io/address/0x0DABC7Fc9cBe23bD26e2ef05bb8e055bE2a58F49>

2.1.2 Documentation



Documentation was not required as the code was self-sufficient for understanding the project.

2.1.3 Audit Goals

CredShields employs a combination of in-house tools and thorough manual review processes to deliver comprehensive smart contract security audits. The majority of the audit involves manual inspection of the contract's source code, guided by OWASP's Smart Contract Security Weakness Enumeration (SCWE) framework and an extended, self-developed checklist built from industry best practices. The team focuses on deeply understanding the contract's core logic, designing targeted test cases, and assessing business logic for potential vulnerabilities across OWASP's identified weakness classes.

CredShields aligns its auditing methodology with the [OWASP Smart Contract Security](#) projects, including the Smart Contract Security Verification Standard (SCSVS), the Smart Contract Weakness Enumeration (SCWE), and the Smart Contract Secure Testing Guide (SCSTG). These frameworks, actively contributed to and co-developed by the CredShields team, aim to bring consistency, clarity, and depth to smart contract security assessments. By adhering to these OWASP standards, we ensure that each audit is performed against a transparent, community-driven, and technically robust baseline. This approach enables us to deliver structured, high-quality audits that address both common and complex smart contract vulnerabilities systematically.

2.2 Retesting Phase

SeerDEX is actively partnering with CredShields to validate the remediations implemented towards the discovered vulnerabilities.

2.3 Vulnerability classification and severity



CredShields follows OWASP's Risk Rating Methodology to determine the risk associated with discovered vulnerabilities. This approach considers two factors - Likelihood and Impact - which are evaluated with three possible values - **Low**, **Medium**, and **High**, based on factors such as Threat agents, Vulnerability factors, and Technical and Business Impacts. The overall severity of the risk is calculated by combining the likelihood and impact estimates.

Overall Risk Severity				
Impact	HIGH	● Medium	● High	● Critical
	MEDIUM	● Low	● Medium	● High
	LOW	● None	● Low	● Medium
		LOW	MEDIUM	HIGH
Likelihood				

Overall, the categories can be defined as described below -

1. Informational

We prioritize technical excellence and pay attention to detail in our coding practices. Our guidelines, standards, and best practices help ensure software stability and reliability. Informational vulnerabilities are opportunities for improvement and do not pose a direct risk to the contract. Code maintainers should use their own judgment on whether to address them.

2. Low

Low-risk vulnerabilities are those that either have a small impact or can't be exploited repeatedly or those the client considers insignificant based on their specific business circumstances.



3. Medium

Medium-severity vulnerabilities are those caused by weak or flawed logic in the code and can lead to exfiltration or modification of private user information. These vulnerabilities can harm the client's reputation under certain conditions and should be fixed within a specified timeframe.

4. High

High-severity vulnerabilities pose a significant risk to the Smart Contract and the organization. They can result in the loss of funds for some users, may or may not require specific conditions, and are more complex to exploit. These vulnerabilities can harm the client's reputation and should be fixed immediately.

5. Critical

Critical issues are directly exploitable bugs or security vulnerabilities that do not require specific conditions. They often result in the loss of funds and Ether from Smart Contracts or users and put sensitive user information at risk of compromise or modification. The client's reputation and financial stability will be severely impacted if these issues are not addressed immediately.

6. Gas

To address the risk and volatility of smart contracts and the use of gas as a method of payment, CredShields has introduced a "Gas" severity category. This category deals with optimizing code and refactoring to conserve gas.

2.4 CredShields staff

The following individual at CredShields managed this engagement and produced this report:

- Shashank, Co-founder CredShields shashank@CredShields.com

Please feel free to contact this individual with any questions or concerns you have about the engagement or this document.



3. Findings Summary -----

This chapter contains the results of the security assessment. Findings are sorted by their severity and grouped by asset and OWASP SCWE classification. Each asset section includes a summary highlighting the key risks and observations. The table in the executive summary presents the total number of identified security vulnerabilities per asset, categorized by risk severity based on the OWASP Smart Contract Security Weakness Enumeration framework.

3.1 Findings Overview

3.1.1 Vulnerability Summary

During the security assessment, 2 security vulnerabilities were identified in the asset.

Vulnerability Title	Severity	Vulnerability Type	Status
Outdated Pragma	Low	Floating Pragma (SCWE-060)	Acknowledged
Dead Code	Informational	Code With No Effects (SCWE-062)	Fixed

Table: Findings and Remediations



5. Bug Reports -----

Bug ID #L001[Acknowledged]

Outdated Pragma

Vulnerability Type

Floating Pragma ([SCWE-060](#))

Severity

Low

Description

Locking the pragma helps ensure that the contracts do not accidentally get deployed using an older version of the Solidity compiler affected by vulnerabilities.

The contract allowed floating or unlocked pragma to be used, i.e., 0.8.32. This allows the contracts to be compiled with all the solidity compiler versions above the limit specified. The following contracts were found to be affected -

Affected Code

- [Token.sol#L3](#)

Impacts

If the smart contract gets compiled and deployed with an older or too recent version of the solidity compiler, there's a chance that it may get compromised due to the bugs present in the older versions or unidentified exploits in the new versions.

Incompatibility issues may also arise if the contract code does not support features in other compiler versions, therefore, breaking the logic.

The likelihood of exploitation is low.

Remediation

Keep the compiler versions consistent in all the smart contract files. Do not allow floating pragmas anywhere. It is suggested to use the 0.8.34 pragma version

Reference: <https://scs.owasp.org/SCWE/SCSVS-CODE/SCWE-060/>

Retest



Client comment : The chosen version 0.8.32, doesn't have any known bugs that will affect the contract. So I believe we can let it go for now.



Bug ID#I001[Fixed]

Dead Code

Vulnerability Type

Code With No Effects ([SCWE-062](#))

Severity

Informational

Description

It is recommended to keep the production repository clean to prevent confusion and the introduction of vulnerabilities. The functions and parameters, contracts, and interfaces that are never used or called externally or from inside the contracts should be removed when the contract is deployed on the mainnet.

The [SeerDEXToken](#) contract inherits from [Ownable2Step](#) (which extends [Ownable](#)) and sets an initial owner in the constructor. However, the [onlyOwner](#) modifier is never applied to any function within the contract, and no ownership-restricted functionality (such as minting, pausing, blacklisting, or fee management) exists. As a result, the entire access control inheritance chain including [transferOwnership\(\)](#), [acceptOwnership\(\)](#), and [renounceOwnership\(\)](#) serves no operational purpose and constitutes dead code.

Affected Code

- [Token.sol#L9](#)

Impacts

This does not impact the security aspect of the Smart contract but prevents confusion when the code is sent to other developers or auditors to understand and implement.

This reduces the overall size of the contracts and also helps in saving gas.

Remediation

If the library functions are not supposed to be used anywhere, consider removing them from the contract.

Retest

This has been fixed.



6. The Disclosure -----

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or team and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.



Your **Secure Future** Starts Here



At CredShields, we're more than just auditors. We're your strategic partner in ensuring a secure Web3 future. Our commitment to your success extends beyond the report, offering ongoing support and guidance to protect your digital assets



 SCS Advocates

